

PROJECT_CONTEXT.md

Project: Peregri (working title)

Status: MVP with the core marketplace loop implemented end-to-end (listings, messaging, match confirmation, delivery handover/receipt, payments, reviews, reports). Still pre-launch; no real users yet.

Purpose of this document: This file is the single source of truth for the project's philosophy, architecture, design language and development history. Every developer or AI agent working on this project should read this document before making changes.

1. Introduction

What is Peregri?

Peregri is a marketplace that connects international travelers with people who need items transported across borders.

Instead of using traditional courier companies, the platform allows ordinary travelers to carry items in unused luggage space and earn money, while requesters receive significantly cheaper international delivery.

The platform is not intended to compete with DHL, FedEx or UPS in logistics.

Instead, it creates a marketplace around trips that are already happening.

Example:

- Alice is flying from Berlin to New York.
- Bob wants someone to bring him a graphics tablet.
- Alice has spare luggage capacity.
- Bob pays Alice through the platform.
- Both users verify completion.
- Both leave reviews.

The traveler earns money for a trip they were already making.

The requester receives a much cheaper delivery.

Everyone wins.

The problem being solved

International shipping is expensive. Sometimes absurdly expensive.

Many people already travel between these places every day with empty suitcase capacity.

That unused capacity has value. Peregri exists to unlock that value.

The platform transforms unused luggage space into a marketplace.

Long-term vision

The long-term goal is not simply "deliver packages."

The vision is to become the default place people visit whenever they think: "Someone is already going there."

Instead of asking friends, instead of posting on Facebook, instead of asking Telegram groups, people should naturally think: "I'll check Peregri."

Eventually the platform should support: travelers, delivery requests, personal shopping, airport pickups, local errands, document delivery, international communities.

Everything connected through one marketplace.

Core philosophy

The project is built around one simple observation: people travel anyway.

Peregri simply connects people who already have matching interests.

The platform does not create transportation. It matches existing transportation.

That distinction is extremely important because it keeps the marketplace scalable and simple.

Target audience

The initial audience is not "everyone." The first users should come from routes where international travel is already common.

Examples: Mexico -> United States, EU <-> UK, Turkey <-> Germany, Poland <-> Ukraine, Azerbaijan <-> Russia, Kazakhstan <-> Russia.

These routes already contain large communities that constantly send items to friends and family.

Launching in one dense corridor is significantly better than launching globally with no liquidity.

Marketplace philosophy

The platform only succeeds if there is enough activity. Unlike social media, an empty marketplace has almost no value. Therefore growth strategy is focused on density rather than geography.

One active route with thousands of users is better than fifty inactive countries.

Every product decision should increase marketplace liquidity.

Business model

Primary revenue source: platform fee on successful transactions.

Potential future revenue: promoted listings, verified traveler subscription, insurance, premium visibility, business accounts, API integrations.

The platform should avoid excessive monetization during the MVP stage. Growth is more valuable than immediate profit.

Trust philosophy

Trust is the single biggest challenge. Every feature should increase trust.

Examples: verified identity, reviews, ratings, completed deliveries, profile history, response time, successful transaction count.

Users are trusting strangers. Everything should communicate safety without creating unnecessary friction.

Product principles

Every feature should satisfy at least one of these goals: reduce friction, increase trust, improve matching, simplify communication.

If it doesn't improve one of those four things, reconsider whether it belongs in the MVP.

What Peregri is NOT

Peregri is NOT: a courier company, a logistics company, Uber for packages, another classified ads website.

Instead it is: a marketplace connecting travelers with delivery requests.

The traveler is always an independent user. The platform only facilitates matching, communication and payment.

Development philosophy

The project intentionally prioritizes: correct architecture over fast hacks.

It is acceptable to spend extra time designing a clean solution if it avoids technical debt.

Quick fixes that make future development harder should be avoided.

The goal is not simply finishing the MVP. The goal is building a foundation that can survive years of development.

AI collaboration philosophy

AI is used as a development assistant. AI should accelerate implementation. AI should NOT redesign architecture without explicit approval.

Whenever AI proposes changes it should preserve: existing architecture, design language, naming conventions, database philosophy, user experience.

Consistency is valued more than novelty. Large rewrites are discouraged unless explicitly requested.

Current project state

The core marketplace loop is implemented end-to-end:

- Authentication (Supabase Auth)
- Profiles (with public profile pages and transaction history)
- Listings (trip / delivery / request, with browse + filtering)
- Messaging (threads, conversations, system-generated notices)
- Match confirmation flow (both parties confirm, courier handover, recipient receipt)
- Completed-deal history, used to drive reviews and profile history
- Reviews (tied to a specific completed deal)
- Payments (Stripe Checkout, webhook-confirmed)
- Reports (user/listing abuse reports)
- A styled 404 page and general app shell/navigation

This section should be re-read and updated whenever a major feature area changes state, so it doesn't drift from the actual codebase the way earlier revisions of this document did.

This document continues with:

1. Design Philosophy
2. UI System
3. Frontend Architecture
4. Backend Architecture
5. Database Design
6. Development History
7. AI Rules
8. Future Roadmap

Those sections are equally important and should always be kept synchronized with the codebase.